**Week 7 All Skills Extra Credit Worksheet**

Write a function called squares_string. It takes an integer argument. Use a for loop to build and return a string representing the squares of the numbers between 1 and the argument, inclusive, each pair separated by a comma and a space. Rewrite using a whileloop.

Write the function oldest(name1, age1, name2, age2, name3, age3). The name parameters are strings, the ages are integers. Return the name of the oldest person.

Write a function called `second_to_last_chars`. Get strings from the user until the user enters the empty string (i.e. hits Enter without typing anything first). Assume all other strings will have at least 2 characters. Return a string that consists of the second to last characters of the strings.

Write a function called `roulette`. It takes one integer argument, the maximum allowed bet. It allows the user to spin the wheel until he/she decides to quit. It returns the user's net profit/loss. The user may bet on a single number or on odd or on even. In a loop get an amount the user wants to bet. In the prompt, include the maximum allowed bet. Get the bet – `'00'`, `'0'`, `'1'`,… `'36'`, `'odd'`, `'even'`. Get a random number between `0` and `37` (`37` for `'00'`) using `random.randint`. Print the result. If the user bet on a number and wins, increase his/her total by 35 * the amount bet. If not, decrease by the amount bet. In the case of odd/even, 0 and 00 always count as a loss. If the user bet on odd/even and wins, increase his/her total by the amount bet. If not, decrease by the amount bet. Print the user's total so far.

Given that the following variable assignments have been executed:

```
time = '5:30 pm'
date = "1009 "
gain = 9
over = True
```

Evaluate each of the following Python expressions. Evaluate the expression and write its value as a literal (remember strings have quotes and booleans are capitalized) in the **Value** column and its type in the **Type** column.

| Expression | Value | Type |
|---|---|---|
| `time[len(time) - 1] * 2` | | |
| `(time + date) * 2` | | |
| `int(date) < gain or not over` | | |
| `11.0 % gain` | | |
| `11.0 // gain` | | |
| `18 / gain` | | |
| `18 // 2 * gain` | | |
| `time[1] + '-' * 3 + date[3]` | | |
| `date + '-' + str(gain)` | | |